

Fees and Receipt Service - Full Project Documentation

Fees and Receipt Service – Full Project Documentation

A Spring Boot API for managing student invoices, idempotent payments, computed balances, PDF receipts, currency-aware reconciliation reporting, and HMAC-verified mock payment gateway webhooks.

1. Overview

This service exists to solve a specific, high-stakes problem: tracking money owed and paid by students in a way that is **correct under retries, correct under concurrent access, and auditable after the fact**. Every design decision in this project traces back to one of those three goals.

Core guarantees this system provides:

- The same payment request, sent twice (accidentally or as a network retry), never creates a duplicate charge.
- Invoice totals are always derived from actual line items – never trusted blindly from a client.
- A payment can never exceed an invoice's outstanding balance.
- A payment's currency must match its invoice's currency – no silent cross-currency corruption.
- Every payment-affecting operation is wrapped in a database transaction – partial writes cannot happen.
- Balances are always computed fresh, never stored – they cannot drift out of sync with reality.

2. Technology Stack

Area	Choice
Core	Spring Boot 3, Spring Web
Language	Java 21 (target; developed on a Java 26 JDK)
Build	Maven
Database	PostgreSQL
Persistence	Spring Data JPA / Hibernate
Migrations	Flyway
PDF receipts	Thymeleaf (HTML templates) + Flying Saucer (bundles OpenPDF)
API docs	Springdoc OpenAPI / Swagger UI
Validation	Jakarta Bean Validation (spring-boot-starter-validation)

3. Project Structure

```
src/main/java/com/felxisaf/feereceiptservice/
|
|--- common/
|   `--- exception/
|       |--- ApiError.java
|       |--- ResourceNotFoundException.java
|       |--- DuplicateResourceException.java
|       |--- BusinessException.java
|       |--- IdempotencyConflictException.java
|       |--- InvalidWebhookSignatureException.java
|       `--- GlobalExceptionHandler.java
|
|--- students/
|   |--- Student.java
|   |--- StudentRepository.java
|   |--- StudentService.java
|   |--- StudentController.java
|   `--- dto/
|       |--- StudentRequest.java
|       |--- StudentResponse.java
|       `--- StudentBalanceResponse.java
|
|--- invoice/
|   |--- Invoice.java
|   |--- InvoiceItem.java
|   |--- InvoiceStatus.java
|   |--- InvoiceRepository.java
|   |--- InvoiceService.java
|   |--- InvoiceController.java
|   |--- Payment.java
|   |--- PaymentRepository.java
|   |--- PaymentService.java
|   |--- PaymentController.java
|   |--- ReceiptService.java
|   `--- dto/
|       |--- InvoiceItemRequest.java
|       |--- InvoiceRequest.java
|       |--- InvoiceItemResponse.java
|       |--- InvoiceResponse.java
|       |--- InvoiceBalanceResponse.java
|       |--- PaymentRequest.java
|       `--- PaymentResponse.java
|
|--- reports/
|   |--- ReportService.java
|   |--- ReportController.java
|   `--- dto/
|       `--- ReconciliationReportResponse.java
|
|--- webhooks/
|   |--- WebhookService.java
|   |--- WebhookController.java
|   `--- dto/
|       `--- WebhookPayload.java

src/main/resources/
|--- application.yml                (committed -- non-secret config)
|--- application-local.yml          (gitignored -- real DB password +
webhook secret)
|--- application-local.example.yml
|--- templates/
|   `--- receipt.html              (Thymeleaf receipt template)
|--- db/migration/
```

```
    `--- V1__init_schema.sql

src/test/java/com/felxisaf/feereceiptservice/
`--- PaymentFlowIntegrationTest.java
```

common, students, invoice, reports, and webhooks are **siblings** – none nested inside another.

4. Database Schema

```
students (id, student_number, full_name, email, created_at)

invoices (id, invoice_number, student_id -> students.id, currency,
total_amount,
          status, due_date, issued_at, created_at)

invoice_items (id, invoice_id -> invoices.id, description, quantity,
               unit_amount, line_total)

payments (id, payment_reference, invoice_id -> invoices.id, amount,
currency,
          payment_method, gateway_reference, idempotency_key,
checksum,
          paid_at, created_at)
```

Key constraints enforced at the database level (not just in Java): -
total_amount > 0, amount > 0 – CHECK constraints - status IN
('DRAFT', 'ISSUED', 'PARTIALLY_PAID', 'PAID', 'VOID') – CHECK
constraint - payment_reference, idempotency_key, email, student_number,
invoice_number – all UNIQUE

Managed entirely by Flyway (V1__init_schema.sql), applied
automatically on application startup.

5. Entity Relationships

- A Student has many Invoices.
 - An Invoice has many InvoiceItems (cascade delete – deleting an invoice deletes its items).
 - An Invoice has many Payments (a payment always references exactly one invoice).
-

6. Invoice Lifecycle

An invoice is never just data – it moves through a defined state machine, and each transition is guarded:

- Every invoice starts as DRAFT – line items and totals can still change conceptually, and it cannot be paid yet.
- POST /invoices/{id}/issue moves it to ISSUED – this is the explicit signal that the invoice is finalized and now owed. Attempting to issue an already-issued invoice is rejected (422).
- Payments are only accepted against ISSUED or PARTIALLY_PAID

- invoices – attempting to pay a DRAFT invoice is rejected (422).
- The invoice automatically becomes PARTIALLY PAID or PAID after each successful payment, based on the total paid so far versus the invoice total.
- VOID exists in the schema for a cancelled invoice, though no endpoint currently transitions an invoice into this state.

7. API Reference

Students

Method	Path	Description
POST	/students	Create a student
GET	/students/{id}	Fetch a student
GET	/students/{id}/balance	Aggregated balance across all their invoices

Invoices

Method	Path	Description
POST	/invoices	Create a draft invoice with line items (total computed server-side)
GET	/invoices/{id}	Fetch an invoice with its items
POST	/invoices/{id}/issue	Transition DRAFT -> ISSUED
GET	/invoices/{id}/balance	Total, paid, and outstanding for one invoice

Payments

Method	Path	Description
POST	/payments	Record a payment (idempotent – see Section 8)
GET	/payments/{id}	Fetch a payment
GET	/payments/{id}/receipt	Download a PDF receipt

Reports

Method	Path	Description
GET	/reports/reconciliation?startDate=&endDate=	Payments in range, grouped by currency, flagged reconciled/unreconciled

Webhooks

Method	Path	Description
		Mock gateway

POST	/webhooks/gateway	callback confirming a payment (HMAC-verified)
------	-------------------	---

Full interactive documentation is available at <http://localhost:8080/swagger-ui.html> once the app is running.

8. Idempotent Payments – How It Works

This is the project's central pain-point.

1. The client sends `POST /payments` with an **Idempotency-Key** HTTP header (a unique string the client generates per logical payment attempt) and a JSON body with `paymentReference`, `invoiceId`, `amount`, `currency`, `paymentMethod`, `gatewayReference` (optional).
2. The server computes a **SHA-256 checksum** of the meaningful request fields (not including the idempotency key itself).
3. It looks up whether this `idempotencyKey` has been used before:
 - **Not found** -> proceed to validation and recording (below).
 - **Found, same checksum** -> this is a safe retry (e.g. a network timeout caused the client to resend). Return the **original** payment; do not create a new row.
 - **Found, different checksum** -> the client reused a key for a genuinely different request. Reject with `409 Conflict`.
4. Independent of idempotency, a duplicate `paymentReference` is always rejected with `409`.
5. The invoice must be `ISSUED` or `PARTIALLY_PAID` otherwise `422`.
6. The payment's `currency` must match the invoice's `currency` – otherwise `422`.
7. The payment amount must not exceed the invoice's outstanding balance – otherwise `422`.
8. If all checks pass: insert the payment row, recompute the invoice's status (`PARTIALLY_PAID` or `PAID`), and save both – **in a single @Transactional method**, so either both writes succeed or neither does.

9. Business Rules Enforced

Rule	Where enforced	HTTP status on violation
Duplicate email / student number	StudentService	409
Duplicate invoice number	InvoiceService	409
Invoice total must be > 0	InvoiceService	422
Only DRAFT invoices can be issued	InvoiceService	422
Duplicate payment reference	PaymentService	409
Idempotency key reused with different body	PaymentService	409
Payment currency must match invoice currency	PaymentService	422
Payment must not exceed outstanding balance	PaymentService	422
Payments only accepted on ISSUED/PARTIALLY_PAID invoices	PaymentService	422

Webhook signature must be valid	WebhookService	401
Malformed / unparseable JSON request body	GlobalExceptionHandler	400
Bean validation failures (blank fields, bad email, etc.)	Jakarta Validation + GlobalExceptionHandler	422
All monetary values	BigDecimal throughout	–

10. PDF Receipts

- Built with a Thymeleaf template (`templates/receipt.html`), written as **strict XHTML** (Flying Saucer's requirement – not loose HTML5).
 - Layout uses tables, not flexbox/grid, since Flying Saucer only supports a CSS 2.1 subset.
 - `ReceiptService` renders the template to an HTML string, then converts it to PDF bytes via Flying Saucer's `ITextRenderer`.
 - Shows student details, invoice line items, this payment, total paid to date, and outstanding balance.
-

11. Reconciliation Reports

- A payment is considered **reconciled** if it has a non-blank `gatewayReference`; else it is **unreconciled**.
 - Totals are grouped **per currency** (`Map<String, BigDecimal>`), never summed together – a direct lesson carried over from the currency bug (Section 13).
 - Day boundaries for the date range are currently computed in **UTC** – a known simplification (Section 14).
-

12. Mock Gateway Webhooks

- `POST /webhooks/gateway` simulates a real payment gateway (e.g. Stripe, Paystack) confirming a transaction after the fact. No real gateway integration exists in this project – this endpoint is the receiving side of that pattern, triggered manually during testing in place of a real gateway.
 - The request body is read as a **raw string**, not auto-parsed, because HMAC verification must operate on the exact bytes sent.
 - Signature verification: `HMAC-SHA256(rawBody, sharedSecret)`, hex-encoded, sent in the `X-Webhook-Signature` header, compared using `MessageDigest.isEqual` (constant-time, to avoid timing attacks).
 - On success: looks up the payment by `paymentReference` and fills in its `gatewayReference` – which is exactly what makes it show as “reconciled” in the reconciliation report.
 - The shared secret lives in `application-local.yml` under `app.webhook.secret` (gitignored) – `application.yml` only holds a `CHANGE_ME` placeholder.
 - **Testing note:** this endpoint cannot be exercised through Swagger UI alone, since the signature must be computed outside the browser first, then pasted in as a header alongside the exact same body text.
-

13. Bug Case Study 1: The Currency Mismatch

Early in testing, a payment was recorded in NGN against an invoice created in USD, and the code accepted it – nothing checked that the two currencies matched. The invoice’s balance calculation then summed the raw numeric amounts together as if $1 \text{ USD} = 1 \text{ NGN}$, producing a total that looked plausible but was financially meaningless.

The fix: `PaymentService` now explicitly rejects a payment whose currency doesn’t match its invoice’s currency (422). The same lesson was carried forward into the reconciliation report, which groups totals by currency rather than summing them.

The broader lesson: a financial system that “doesn’t crash” is not the same as a financial system that’s correct. Silent numeric corruption is far more dangerous than an exception, because it looks fine until someone tries to reconcile the books.

14. Bug Case Study 2: Success Reported as Failure (LazyInitializationException)

While testing `POST /invoices/{id}/issue`, the database correctly showed the invoice’s status updated to `ISSUED` – yet the client still received a 500 Internal Server Error.

Root cause: `InvoiceService.issueInvoice(...)` is `@Transactional`. Spring commits that transaction and closes the underlying database session the moment the method returns control to the controller. The controller then immediately calls `InvoiceResponse.fromEntity(invoice)`, which iterates `invoice.getItems()` – a **lazily-loaded** collection that Hibernate had not yet fetched from the database. Since the session was already closed, Hibernate had no way to retrieve it, and threw `LazyInitializationException`. The generic exception handler correctly caught this and returned a 500 – but by then, the actual database write had already succeeded and committed. The client was told the operation failed when it had, in fact, worked.

The fix: `InvoiceRepository` gained a new method using `LEFT JOIN FETCH` to load an invoice and its line items in a single query, ensuring the collection is fully populated *before* the transaction closes:

```
@Query("SELECT i FROM Invoice i LEFT JOIN FETCH i.items WHERE i.id = :id")
Optional<Invoice> findByIdWithItems(@Param("id") Long id);
```

`InvoiceService.getInvoiceById(...)` now uses this method, which transitively fixed both `GET /invoices/{id}` and `POST /invoices/{id}/issue`, since `issueInvoice(...)` calls `getInvoiceById(...)` internally.

The broader lesson: a successful database write and a successful HTTP response are two different things, and a bug can break the connection between them. When a client-reported failure doesn’t match what’s actually in the database, the discrepancy itself is the clue – check what happened *after* the write, not just the write logic itself.

15. Known Limitations / Open Items

- **No multi-currency support.** An invoice has exactly one currency; a student's aggregated balance assumes all their invoices share a currency.
 - **Reconciliation report day boundaries use UTC**, not an institution-configurable timezone.
 - **No real payment gateway integration.** The webhook endpoint is a mock/simulation of the pattern a real integration (Stripe, Paystack, etc.) would use – no outbound calls to any real gateway exist in this project.
 - **Security cleanup pending:** real PostgreSQL credentials were at one point committed directly in the tracked `application.yml`. The recommended fix is to (a) scrub the credentials from `application.yml` going forward, and (b) rotate the actual PostgreSQL password, since anything already pushed to GitHub history should be treated as permanently exposed regardless of later edits.
 - **Finalized OpenAPI/Swagger annotations** (descriptions, examples) have not yet been polished.
-

16. Local Setup

1. Copy the example config:

```
Copy-Item .\src\main\resources\application-local.example.yml
.\src\main\resources\application-local.yml
```

2. Edit `application-local.yml` – set your real PostgreSQL password and a random webhook secret.
3. Run with the `local` profile:

```
mvn spring-boot:run "-Dspring-boot.run.profiles=local"
```

4. Swagger UI: <http://localhost:8080/swagger-ui.html>
-

17. Testing the Full Flow Manually

1. POST `/students` -> create a student
2. POST `/invoices` -> create an invoice with at least one line item
3. POST `/invoices/{id}/issue` -> move it to ISSUED
4. POST `/payments` with header `Idempotency-Key: test-key-1` -> record a payment
5. Repeat step 4 exactly -> same payment returned, not a new one
6. Same key, different amount -> 409
7. Amount exceeding balance -> 422
8. Mismatched currency -> 422
9. GET `/invoices/{id}/balance` and GET `/students/{id}/balance` -> confirm totals
10. GET `/payments/{id}/receipt` -> PDF downloads
11. Compute an HMAC signature and call POST `/webhooks/gateway` -> `gatewayReference` gets filled in
12. GET `/reports/reconciliation?startDate=...&endDate=...` -> that payment shows as reconciled

An automated version of this flow, plus dedicated regression tests for both bugs in Sections 13 and 14, lives in

`PaymentFlowIntegrationTest.java`.
